

2. Testing Throughout the SDLC

A. SDLC models

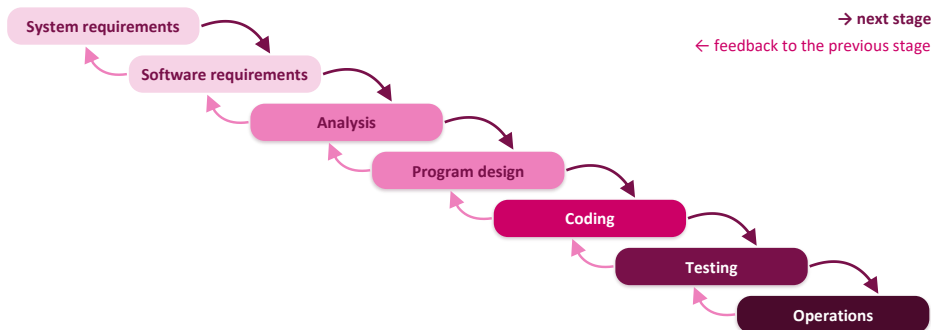


A **software development lifecycle (SDLC) model** is an abstract, high-level representation of the development process: how its phases and activities relate, logically and chronologically.

- **Sequential:** e.g., waterfall, V-model
- **Iterative:** e.g., spiral model, prototyping
- **Incremental:** e.g., Unified Process

Activities can also be described by more detailed **methods and Agile practices**: ATDD, BDD, DDD (domain-driven design), XP (Extreme Programming), FDD (feature-driven development), Kanban, Lean IT, Scrum and TDD.

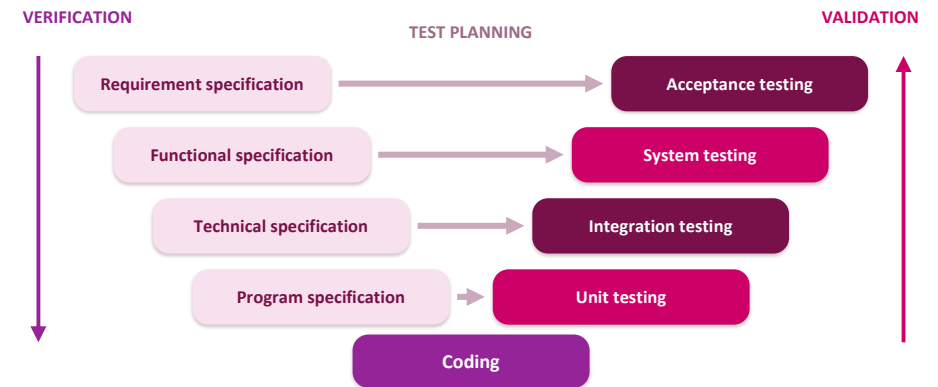
B. Royce's waterfall model (sequential)



Each phase is completed before the next one starts.

- **Pros:** enforces discipline at each stage; defined start and end; progress is easy to identify; requirements come before code, so less wasted time and better quality
- **Cons:** everything is delivered at the end, maybe years later; customers don't always know what they want until they see it; requirements change with business and technology; time and cost are hard to estimate; what's requested can't always be built as wanted

C. The V-model (sequential)

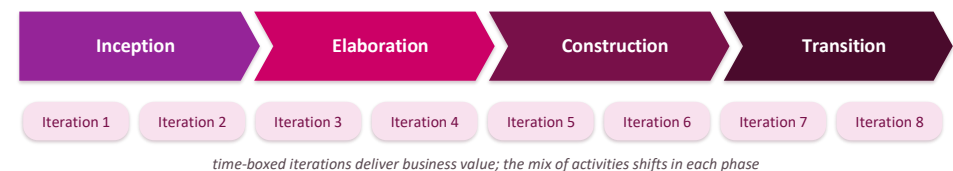


Development runs **down the left** of the V, from requirement specification to coding; testing runs **up the right**, from unit testing to acceptance testing. Each development stage has a **matching test level**, and **test planning** for that level is done during the matching stage (the dashed arrows), so tests are ready early.

- **Pros:** test plans are developed earlier, giving a higher chance of success; defects are found earlier; works well on smaller projects
- **Cons:** quite rigid in execution; no early prototype; test documents must be updated along the way

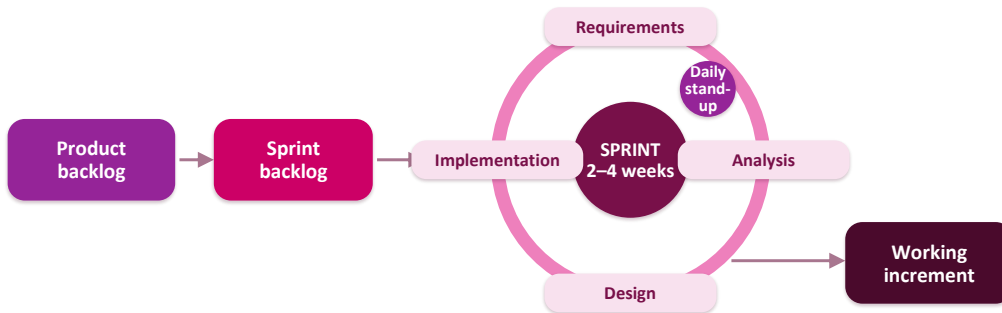
The classic V shows four test levels; the syllabus defines five (section 2.2), with integration split into component integration and system integration.

D. Unified Process (iterative and incremental)



- **Sequential phases:** inception, elaboration, construction, transition
- **Iterative and incremental:** business value is delivered in time-boxed, cross-discipline iterations
- **The balance of activities shifts:** the mix of activities, from business modelling to deployment, changes within each phase

E. Agile methodologies



- **Scrum:** sprints, backlog, daily stand-up, review and retrospective
- **XP:** nested feedback loops, from the release plan down to pair programming
- **Kanban:** visual board; limit work in progress
- **TDD:** red → green → refactor
- **FDD:** model, list features, then plan, design and build by feature
- **Lean:** define value, map the stream, create flow, establish pull
- **BDD:** behaviour described in plain-language scenarios

Pros: fast deployment; small, frequent improvements following feedback; team skills improve. **Cons:** light documentation; formal records of change may not be created; regression testing may get out of control.

F. Impact of the SDLC on testing

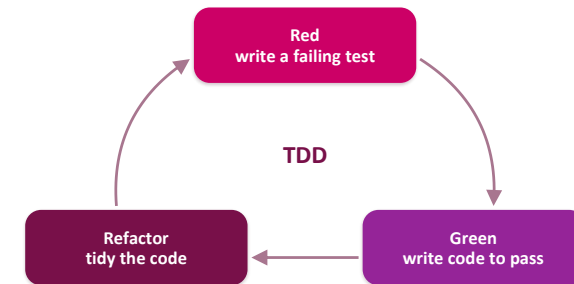
Testing must be **adapted to the SDLC** to succeed. The SDLC affects the **scope and timing** of test activities (test levels and types), the **level of detail** of test documentation, the choice of **test techniques and approach**, the **extent of test automation**, and the **tester's role and responsibilities**.

- **Sequential:** in early phases testers join requirement reviews, test analysis and test design; code arrives late, so dynamic testing comes late. Static testing can happen at any time
- **Iterative and incremental:** each iteration delivers an increment, so static and dynamic testing can happen at all test levels in every iteration; needs fast feedback and extensive regression testing
- **Agile:** change is expected throughout, so documentation is lightweight, there is extensive test automation for regression testing, and manual testing is mostly experience-based

G. Good testing practices for any SDLC

1. Every development activity has a **corresponding test activity**, so all development is subject to quality control
2. Each test level has its **own specific objectives**: comprehensive testing without redundancy
3. Test analysis and design for a level **start during the matching development phase** (early testing)
4. Testers **review work products as soon as drafts exist** (shift left)

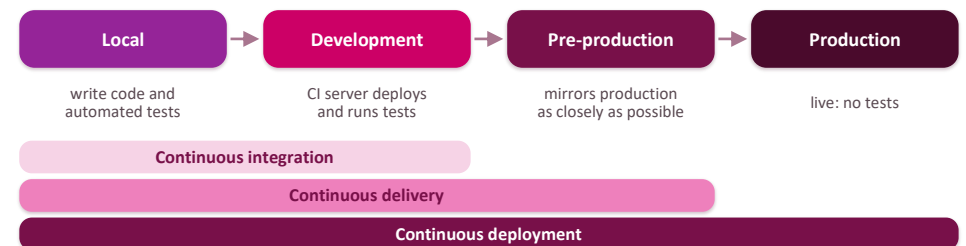
H. Testing as a driver for development



In **TDD, ATDD and BDD**, tests are defined **before** the code, as a way of directing development. They follow early testing and shift left, support iterative development, and the tests can persist as automated tests for future changes and refactoring.

- **TDD** (test-driven development): directs coding through test cases instead of extensive design: write a test, write code to pass it, refactor
- **ATDD** (acceptance test-driven development): derives tests from **acceptance criteria** as part of system design, before that part is built
- **BDD** (behaviour-driven development): expresses desired behaviour as test cases, usually in Gherkin: **Given / When / Then**

I. DevOps and testing

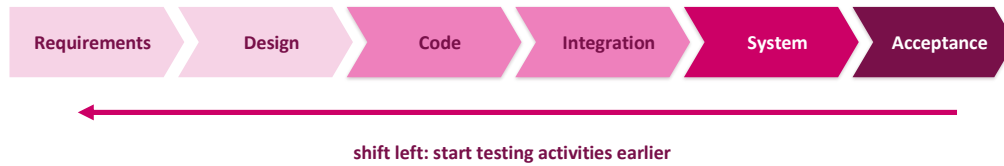


DevOps gets **development (including testing) and operations** working together towards common goals: a cultural shift that values both equally. It promotes team autonomy, fast feedback, integrated toolchains, and **CI and CD**, to build, test and release high-quality code faster through a delivery pipeline. Continuous delivery is automated up to production; continuous deployment also releases to production automatically.

Benefits for testing: fast feedback on code quality and whether changes break existing code; shift left, because developers submit code with component tests and static analysis; stable test environments through automated CI/CD; more focus on non-functional qualities (e.g., performance, reliability); less repetitive manual testing; lower regression risk.

Risks and challenges: the delivery pipeline must be defined and established; CI/CD tools must be introduced and maintained; test automation needs extra resources and can be hard to establish and maintain. Manual testing is **still needed**, especially from the user's perspective.

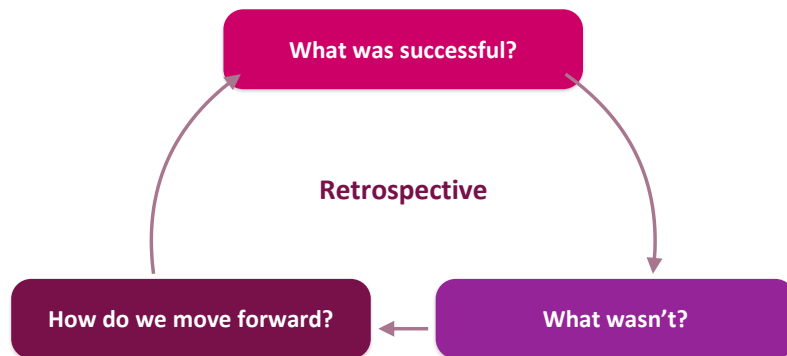
J. Shift left



Shift left is another name for **early testing**: don't wait for code to be written or components to be integrated. It **doesn't** mean neglecting later testing. It may cost more training, effort or money early on but should save more later, and stakeholders must be convinced of the approach.

- **Review specifications as a tester**: often finds ambiguities, gaps and inconsistencies
- **Write test cases before the code**, and run code in a test harness during implementation
- **Use CI and CD**: fast feedback, with automated component tests going in with the code
- **Static analysis first**: before dynamic testing, or as part of an automated process
- **Non-functional testing early**: start at component level

K. Retrospectives



Held at the end of a project or iteration, at a release milestone, or as needed. The **whole team** (not just testers) discusses what was successful and should be kept, what wasn't and could be improved, and how to move forward. Results are recorded, usually in the **test completion report**, and followed up: essential for continuous improvement.

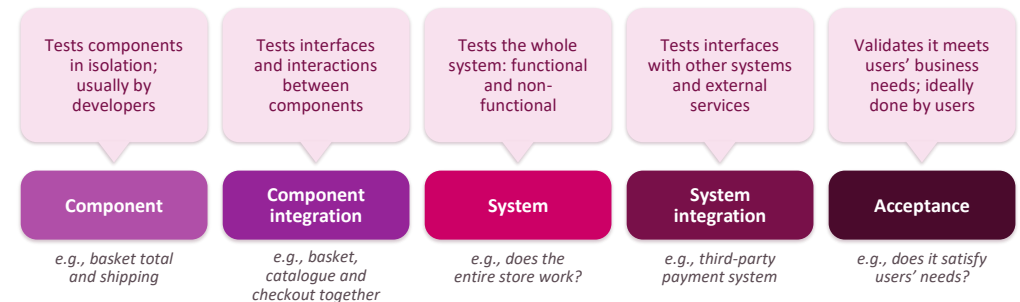
Benefits: test effectiveness and efficiency; better testware; team bonding and learning; a better test basis; better cooperation between development and testing.

L. Test levels vs test types

- **Test levels**: groups of test activities organised and managed together; each is an instance of the test process at a stage of development, from components to systems of systems
- **Test types**: groups of test activities related to specific quality characteristics

In sequential models, one level's exit criteria often feed the next level's entry criteria; in iterative models, activities may span levels and levels may overlap. Levels and types are **two different dimensions**: a level can include many test types.

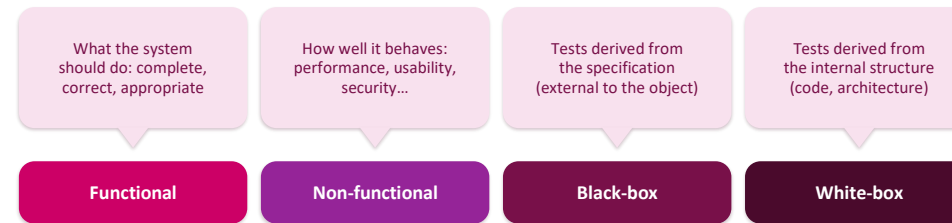
M. The five test levels



- **Component (unit) testing**: tests components **in isolation**; often needs unit test frameworks and test harnesses; normally done by developers in their development environment
- **Component integration (unit integration) testing**: tests the **interfaces and interactions** between components; heavily dependent on the integration strategy: bottom-up, top-down or big-bang
- **System testing**: the **overall behaviour and capabilities** of the entire system: functional testing of end-to-end tasks and non-functional testing (e.g., usability). Some tests need a representative environment, and subsystems may be simulated. Often done by an independent test team
- **System integration testing**: **interfaces with other systems and external services**; needs environments preferably similar to the operational environment, so systems simulated in system testing are now connected for real
- **Acceptance testing**: **validation** and demonstrating readiness for deployment: does the system fulfil the user's business needs? Ideally done by the intended users. Forms: user (UAT), operational, contractual and regulatory acceptance testing, and alpha and beta testing

Each level is distinguished by its **test object, test objectives, test basis, typical defects and failures, and approach and responsibilities**, so test activities don't overlap.

N. Test types



All four test types apply at **every** test level; the focus differs by level, and any test technique can be used to derive their tests.

- **Functional:** what the test object should do; checks functional completeness, correctness and appropriateness
- **Non-functional:** how well the system behaves; checks non-functional quality characteristics
- **Black-box** (specification-based): tests come from documentation external to the test object; objective: check behaviour against its specification
- **White-box** (structure-based): tests come from the implementation or internal structure (code, architecture, workflows, data flows); objective: cover the structure to an acceptable level

Quality characteristics (ISO/IEC 25010): performance efficiency, compatibility, usability, reliability, security, maintainability, portability and safety.

Non-functional testing in practice: start early (e.g., in reviews and component testing); many tests reuse functional tests plus a non-functional check (e.g., response time); late discovery of non-functional defects threatens the project; some tests need special environments (e.g., a usability lab).

O. Confirmation and regression testing



Confirmation testing confirms that an original defect has been fixed. Depending on risk: re-run all tests that previously failed because of the defect, and/or add new tests to cover changes made in the fix. When time is short, just repeat the steps that reproduce the failure and check it no longer occurs.

Regression testing confirms a change (even a confirmed fix) hasn't caused adverse consequences in the changed component, other components, connected systems or the environment. Start with **impact analysis** to see which parts could be affected and size the regression testing.

Regression suites run many times and grow with every iteration or release, so they are **strong candidates for automation**: start early and include them in CI, possibly at several test levels. Confirmation and/or regression testing is needed at every level where fixes or changes are made.

P. Maintenance testing



Categories of maintenance: corrective, adaptive to environment changes, or improving performance or maintainability (ISO/IEC 14764); through planned releases and deployments, or unplanned hot fixes.

Impact analysis can be done before a change, to help decide whether to make it. **Testing in production** checks that the change worked, and looks for regressions in the unchanged parts, usually most of the system.

Scope depends on the degree of risk of the change, the size of the existing system and the size of the change.

- **Modifications:** planned enhancements (release-based), corrective changes, hot fixes
- **Upgrades and migrations:** e.g., moving from one platform to another; test the new environment and the changed software, and data conversion when data is migrated in
- **Retirement:** when an application reaches end of life; test data archiving for long retention periods, and restore and retrieval after archiving